

Task No 5 Implement various searching and sorting operations in python programming.

Date 10/03/25

Aim To implement various searching and sorting operations in python programming.

Ex. A company stores employee records in a list of dictionaries, where each dictionary contains id, name, and department. Write a function find-employee-by-id that takes this list and a target employee ID as arguments and returns the dictionary of the employee with the matching ID, or None if no such employee is found.

Algorithm:

1. Input Definition:

2. Define the function find-employee-by-id that takes two parameters:
a. list of dictionaries (employees), where each dictionary represents an example employee record with key id, name, and department
b. An integer (target-id) representing the employee ID to be searched.

3. Iterate through the list:

use a for loop to iterate through each dictionary in the employees list.

4. Check for matching ID:

within the loop, check if the id field of the current dictionary matches the target-id.

5. Return matching record:

If a match is found, return the current dictionary.

6. Handle no match:

If the loop completes without finding a match, return None.

Output

{ 'id': 2, 'name': 'Bob', 'department': 'engineering' }

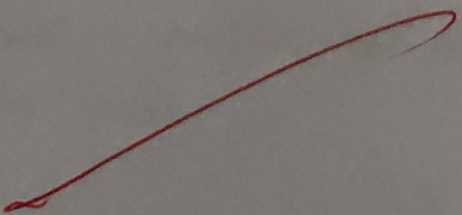


Program:

```
def find-employee-by-id (employees, target-id):  
    for employee in employees:  
        if employee['id'] == target-id:  
            return employee  
    return None.
```

test the function

```
employees = [  
    {'id': 1, 'name': 'Alice', 'department': 'HR'},  
    {'id': 2, 'name': 'Bob', 'department': 'Engineering'},  
    {'id': 3, 'name': 'Charlie', 'department': 'Sales'},  
]  
print(find-employee-by-id(employees, 2)).
```



5.2 You are developing a grade management system for a school. The system maintains a list of student records, where each record is represented as a dictionary containing a student's name and score. The school needs to generate a report that displays student's score in ascending order, your task is to implement a feature that sorts the student records by their scores using the bubble sort algorithm.

Algorithm

1. Initialization:

- Get the length of the students list and store it in n .

2. Outer loop:

- Iterate from $i=0$ to $n-1$ (inclusive). This loop represents the number of passes through the list.

3. Track swaps:

- Initialise a boolean variable `swapped` to `False`. This variable will track if any swaps are made in the current pass.

4. Inner loop:

- Iterate from $j=0$ to $n-i-2$ (inclusive). This loop compares adjacent elements in the list and performs swaps if necessary.

5. Compare and swap:

- For each pair of adjacent elements (i.e., `students[i]` and `students[i+1]`):
- Compare their score values.
- Set `swapped = students[i]['score'] > students[i+1]['score']`, swap two elements.
- Set `swapped` to `True` to indicate that a swap was made.

6. Early Termination:

- After each pass of the inner loop, check if `swapped` is `False`. If no swaps were made during the pass, the list is already sorted, and you can break out of the outer loop early.

7. Completion:

- The function modifies the students list in place, sorting it by score.

Output 1

Before sorting:

```
{ 'name': 'Alice', 'score': 88 }
```

```
{ 'name': 'Bob', 'score': 95 }
```

```
{ 'name': 'Charlie', 'score': 75 }
```

```
{ 'name': 'Diana', 'score': 85 }
```

After sorting:

```
{ 'name': 'Charlie', 'score': 75 }
```

```
{ 'name': 'Diana', 'score': 85 }
```

```
{ 'name': 'Alice', 'score': 88 }
```

```
{ 'name': 'Bob', 'score': 95 }
```


Program:

```
def bubble-sort - scores (students):
```

```
    n = len(students)
```

```
    for i in range(n):
```

```
        # track if any swap is made in this pass.
```

```
        Swapped = False
```

```
        for j in range(0, n-i-1):
```

```
            if students[j]['score'] > students[j+1]['score']:
```

```
                # swap if the score of the current student is greater than next  
                students[j], students[j+1] = students[j+1], students[j]
```

```
                Swapped = True
```

```
            # if no two elements were swapped, the list is already sorted  
            if not swapped:
```

```
                break
```

```
## Example usage.
```

```
students = [
```

```
    { 'name': 'Alice', 'score': 88
```

```
    { 'name': 'Bob', 'score': 95 }
```

```
    { 'name': 'Charlie', 'score': 75 },
```

```
    { 'name': 'Diana', 'score': 65 }  
]
```

```
print("Before sorting:")
```

```
for student in students:
```

```
    print(student)
```

```
bubble-sort-scores(students)
```

```
print("In After sorting:")
```

```
for student in students:
```

```
    print(student).
```

VEL TECH	
EX NO.	5
PERFORMANCE (5)	5
RESULT AND ANALYSIS (5)	5
VIVA VOCE (5)	5
RECORD (5)	
TOTAL (20)	
DATE	15

Result: Thus, the program for various Searching and Sorting operations is executed and verified successfully.